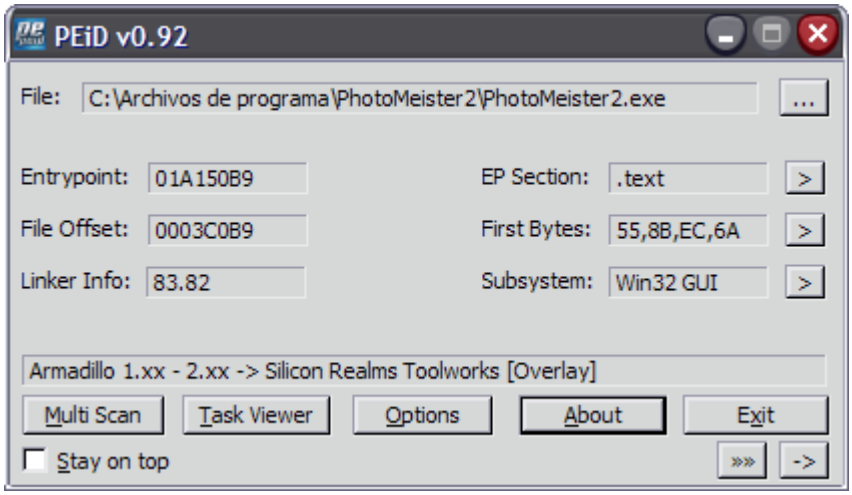
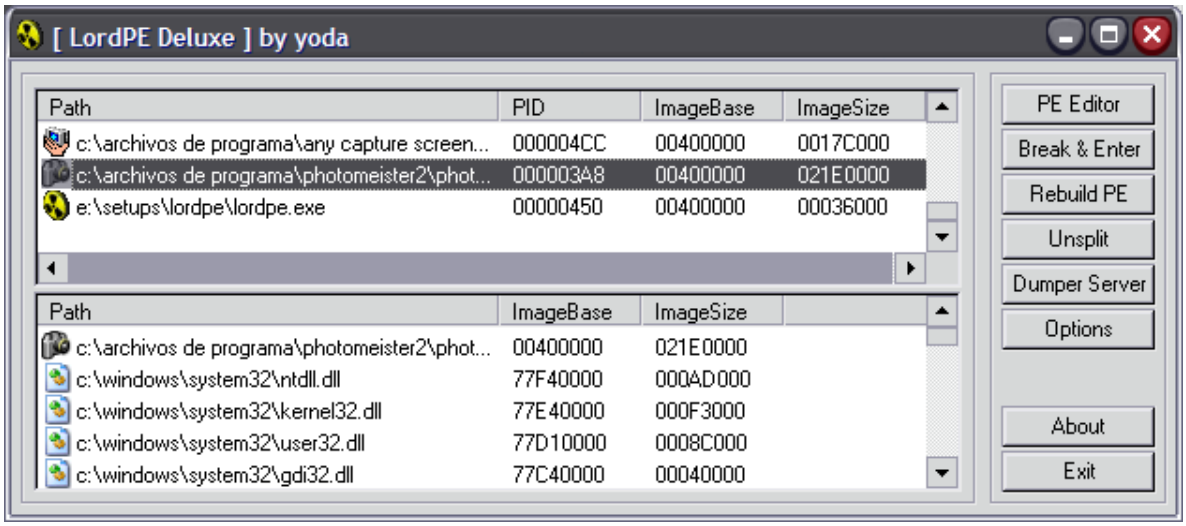
Programa:	PhotoMeister 2.4	
Protección:	Empaquetado con Armadillo 1.xx-2.xx según PEiD	
Objetivos:	Desempaquetar el archivo	
URL descarga:	---	
Dificultad:	Nunca se sabe con el Armadillo	
Herramientas:	OllyDbg + PEiD + LordPE + ImpRec.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 04-08-2005

Hola a todo@s una vez más...!!

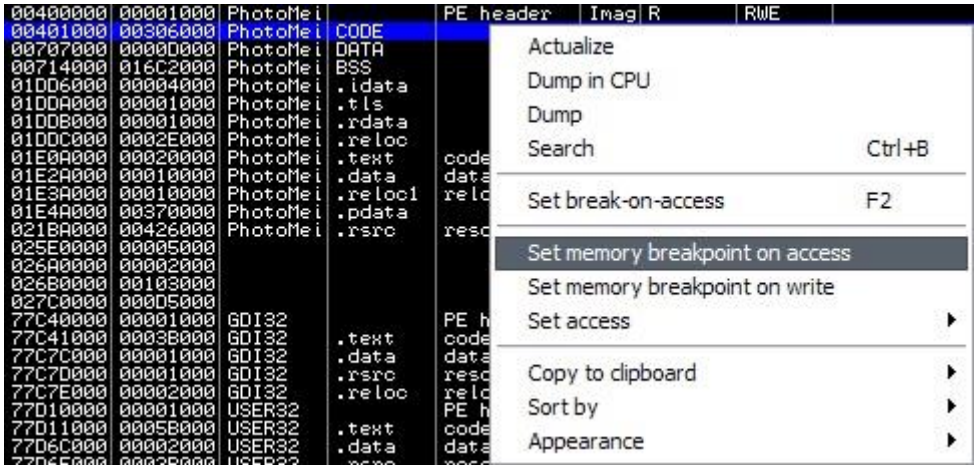
Aqui estamos de nuevo tras el último tuto de CloneCD 2.8.3.1 que nos dió mucho trabajo encarando otra protección, en este caso Armadillo probablemente la versión 2.85 pero no lo puedo garantizar, lo primero de todo y como siempre le preguntaremos a PEiD:



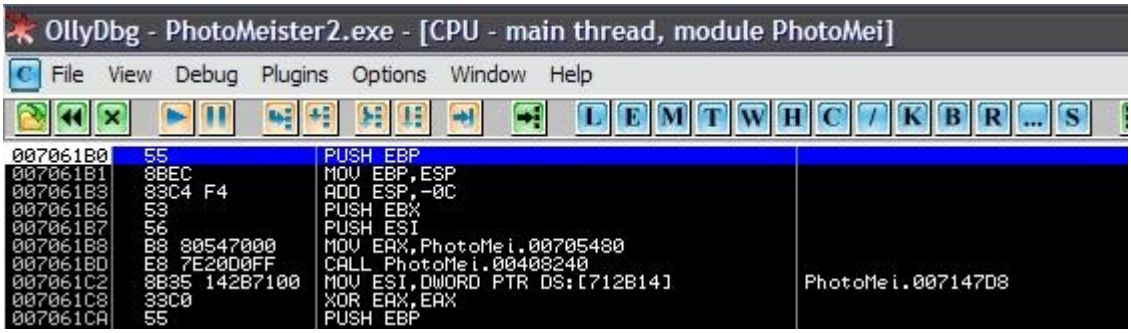
En estas versiones siempre nos dice los mismo y no concreta mucho pero ya nos vale igual, nos quedamos con que es Arma, bien ahora el siguiente paso será ver cuantos procesos se cargan, yo suelo utilizar LordPE, ejecutamos el programa y miramos entre los procesos:



Sólo uno por lo que quedan descartados los Nanomites y por supuesto el CopyMemII, veamos que tiene, carguemos el progama en Olly, parados en su EP ponemos un BPM en la primera seccion,



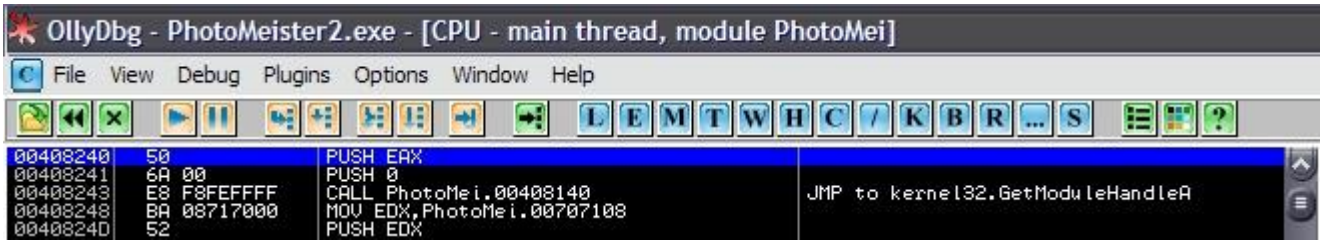
una vez puesto le damos a F9 y para en el OEP:



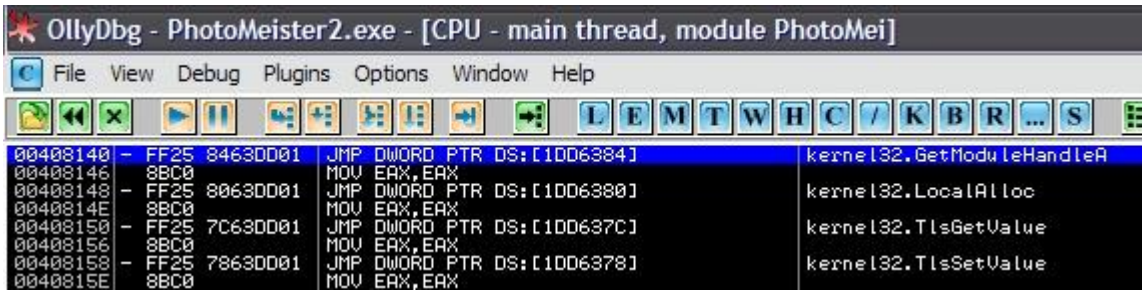
Ya tenemos el primer dato que necesitamos:

OEP: 7061B0

sigamos, ahora busquemos la iat, como esta hecho en Delphi, ya que nos es dificil darse cuenta, sabemos que el primer call va a GetModuleHandleA, entremos en el a ver que hay:



Efectivamente alli esta el call citado, entremos dentro y llegamos a la tabla de saltos,



Vemos punteros que apuntan a zonas de Armadillo con direcciones de tipo [1DD63xx] seleccionamos cualquiera de ellos y hacemos un Follow in Dump, caemos en la iat, ahora buscamos el inicio subiendo hacia arriba:

```
Address   Hex dump                                          ASCII
01DD6238  00 00 00 00 6A 98 9D 01 00 00 00 00 00 00 00 00  ....jÿØØ........
01DD6248  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
01DD6258  DF A7 E5 77 CA 25 F4 77 90 56 F6 77 DE 55 F6 77  ■ØßwÊ%¶wÉV÷wÏU÷w
01DD6268  45 A7 E5 77 CB 15 E6 77 72 AC E5 77 A0 60 E5 77  EØßw╦§µwr¼ßwá'ßw
01DD6278  82 A6 E5 77 71 A6 E5 77 60 A6 E5 77 44 7F E5 77  éªßwqªßw'ªßwDßßw
01DD6288  49 A9 E5 77 AF 9B E4 77 8B 98 E4 77 FC A7 E5 77  I®ßw»øßwïÿßw³ØßW
```

Ahi vemos donde comienza la tabla, ademas tiene un valor correcto, busquemos el final de la misma manera:

```
Address   Hex dump                                          ASCII
01DD6C48  00 00 00 00 11 54 4F 03 31 53 4F 03 EB 52 4F 03  ....◄TO♥1SO♥ûRO♥
01DD6C58  89 52 4F 03 00 00 00 00 A5 A1 B0 76 5C 46 B1 76  ëRO♥....ñ¡▒v\F▒v
01DD6C68  00 00 00 00 00 10 52 03 E0 1D 52 03 20 16 52 03  .....►R♥∟#R♥ ▬R♥
01DD6C78  C0 19 52 03 60 19 52 03 00 00 00 00 A1 39 E5 77  └↓R♥'↓R♥....i9ßw
01DD6C88  00 00 00 00 6B 65 72 6E 65 6C 33 32 2E 64 6C 6C  ....kernel32.dll
```

Tambien tenemos el final, con estos dos datos obtenemos su longitud:

IAT => Inicio: 1DD6258
Final: 1DD6c84
Largo: A30

Vemos ademas que hay punteros redirigidos apuntando hacia zonas del tipo :02D6xxxx y que son zonas de Armadillo, antes de nada recuperemos la iat:

```
01DD62A8  41 DA E5 77 7E 17 E4 77 56 4B D6 02 86 AD E5 77  Aßßw~↨ßwVKî☻§¡ßw
01DD62B8  A9 AD E5 77 62 73 E5 77 02 15 F4 77 58 E3 E5 77  ®¡ßwbsßw☻§¶wXßßw
01DD62C8  1A 51 D6 02 85 61 D6 02 9D EE E5 77 2F 52 D6 02  →Qî☻àaî☻Ø¯ßw/Rî☻
01DD62D8  FB 52 D6 02 26 53 D6 02 AC 58 D6 02 C0 30 E7 77  ¹Rî☻&Sî☻¼Xî☻└0╫w
01DD62E8  C8 58 D6 02 26 C8 E4 77 44 0C F5 77 86 56 D6 02  ╚Xî☻&╠ßwD♀⌡wåVî☻
01DD62F8  37 38 E5 77 94 E4 E5 77 7F 59 D6 02 0C 16 E4 77  78ßwößßw⌂Yî☻♀▬ßw
01DD6308  6E 5A D6 02 C4 53 D6 02 28 5A D6 02 00 00 00 00  nZî☻─Sî☻(Zî☻....
01DD6318  76 8A D3 77 00 6E D1 77 34 62 D6 02 10 A3 D1 77  vèëw.nÐw4bî☻►úÐw
01DD6328  00 00 00 00 10 24 DA 77 9A 22 DA 77 D8 17 DA 77  ....►$ rwÜ" rwï↨rw
01DD6338  00 00 00 00 BE 2E 0F 77 22 30 0F 77 7F 32 0F 77  ....¥.☼w"0☼w⌂2☼w
```

Vamos a seguirle la pista al primer puntero redirigido ***[1DD62A4]=02D6506E*** le ponemos un ***HBPWrite*** y reiniciamos con ***Ctrl+F2***, damos ***RUN*** y ***Olly*** parara una primera vez pero no vemos nada interesante, damos a ***F9*** nuevamente y para aqui:

```
:02D72260   8908            MOV DWORD PTR DS:[EAX],ECX
:02D72262   8B85 74FCFFFF   MOV EAX,DWORD PTR SS:[EBP-38C]
```

justo el lugar donde rellena la tabla y por supuesto redirecciona algunos punteros, ahora nos encontramos en un loop, nuestro siguiente objetivo es encontrar el salto magico que nos repare los punteros invalidos, en estos casos es interesante buscar el call que pone en ***EAX*** (normalmente) en valor de la ***API*** correcta, yo le suelo llamar ***call GetProcAddress*** porque aunque sin serlo realmente si cumpe esta funcion, una vez localizado este call, miraremos un poco antes a ver si existe un salto condicional (***JE***, ***JNE***,..) que de producirse pase por encima del call encontrado, si es asi podemos casi asegurar que hemos localizado el salto magico, no obstante hay que asegurarse, ahora bien, en el caso de que no veamos nada antes del call es que el salto esta dentro e igualmente de bemos buscar, un buen metodo como hace Ricardo es ir anotando en un papel cuando un salto se produce y cuando no, asi con todos, despues de unas cuantas pasadas ya se puede determinar cual es el magico. Y dicho esto vamos a buscar el nuestro que yo tampoco se donde esta ;DDD. Traceemos un poco el loop, rapidamente encontramos el call buscado, es este:

```
:02D721FC   FFB5 64FCFFFF   PUSH DWORD PTR SS:[EBP-39C]
:02D72202   FFB5 88FCFFFF   PUSH DWORD PTR SS:[EBP-378]
:02D72208   E8 4929FFFF     CALL 02D64B56        ;Pone en EAX el valor a guardar en la iat..!!
```

si miramos por arriba no vemos ningun salto condicional cercano como en otros Armadillos asi que no cabe mas que pensar que estara dentro. Ahora es cuestion de tracear un poco, cada uno tiene sus truquitos en estos casos, sobretodo la experiencia cuenta mucho para localizarlo aunque no es imprescindible sino nadie haria el primero ¿no creeis? bueno por si habeis tenido algun que otro problemilla pondre aqui la zona del magico que se parece mucho a otras versiones, hasta hoy creo que habre visto cuatro tipos de magico y estos siempre se han ido repitiendo, el de este caso es el siguiente codigo:

```
:02D64BCB   E8 F05C0100      CALL 02D7A8C0                ;Si retorna EAX=0 habra redireccion ;(
:02D64BD0   59               POP ECX
:02D64BD1   59               POP ECX
:02D64BD2   85C0             TEST EAX,EAX
:02D64BD4   74 0A            JE SHORT 02D64BE0            ;Salto magico..!!
:02D64BD6   83C6 10          ADD ESI,10
:02D64BD9   397E 08          CMP DWORD PTR DS:[ESI+8],EDI
:02D64BDC  75 DD             JNZ SHORT 02D64BBB
:02D64BDE   EB B9            JMP SHORT 02D64B99           ;Debemos salir del bucle por aqui
:02D64BE0   8B46 08          MOV EAX,DWORD PTR DS:[ESI+8]     ;pone en EAX un valor de Armadillo..!!
:02D64BE3   EB C1            JMP SHORT 02D64BA6           ;Y va a la tabla a guardarlo
```

Esta es la zona que decide si se debe redirigir o no la iat, esto en concreto se decide en el ***call 2d7a8c0*** si retorna ***EAX=1*** sigue en el bucle mientras ***[ESI+8] ≠ EDI***, y si retorna ***EAX=0*** va a redirecciona directamente, la solucion pasa por nopear el salto en ***:02d64bd4***, pero como sabemos el Arma suele ser bastante quejoso en este sentido y no nos deja tocar ni un sólo byte, entonces necesitamos un mini_script que nos solucione el tema, pero antes vamos hasta el ***OEP*** y alli ponemos un ***HE***, luego otro en el magico (:02d64bd4) y reiniciamos con ***Ctrl+F2***, luego al dar RUN parara en el magico debido al ***BPX***, este sera el punto gatillo o disparador del script porque es desde aqui donde lo ejecutaremos, y sera el siguiente:

```
----------------------/ script /----------------------
start:
eob break
run

break:
cmp eip, 2d64bd4
jne exit
mov eip, 2d64bd6
jmp start

exit:
ret
----------------------/ script /----------------------
```

Lo que hace es muy simple, cuando pare por una ***excepcion*** o un ***breakpoint*** le pasara el control a la etiqueta ***break***, aqui se mirara el valor del ***EIP***, si esta situado en la direccion del salto magico le cambiamos su valor para que apunte a la siguiente instruccion simulando un nopeo y pasando de nuevo el control a la etiqueta ***start*** para seguir con el ciclo, cuando haya una excepcion o un breakpoint y no sea en el magico es que hemos terminado:

Ahi tenemos el tipico mensajito de cuando un script termina el trabajo, aceptamos y estaremos en el OEP, pero aun no podemos dumpear el archivo ya que no tenemos una cabecera de archivo correcta, veamos esto:

00400000	00001000	PhotoMei			Imag	R	RWE
00401000	00306000	PhotoMei	CODE		Imag	R	RWE
00707000	0000D000	PhotoMei	DATA		Imag	R	RWE
00714000	016C2000	PhotoMei	BSS		Imag	R	RWE
01DD6000	00004000	PhotoMei	.idata		Imag	R	RWE
01DDA000	00001000	PhotoMei	.tls		Imag	R	RWE
01DDB000	00001000	PhotoMei	.rdata		Imag	R	RWE
01DDC000	0002E000	PhotoMei	.reloc		Imag	R	RWE
01E0A000	00020000	PhotoMei	.text	code	Imag	R	RWE
01E2A000	00010000	PhotoMei	.data	data,import	Imag	R	RWE
01E3A000	00010000	PhotoMei	.reloc1	relocations	Imag	R	RWE
01E4A000	00370000	PhotoMei	.pdata		Imag	R	RWE
021BA000	00426000	PhotoMei	.rsrc	resources	Imag	R	RWE

Os explico por encima lo que hace ***Armadillo*** es cambiar el valor de la ***PE*** signature que está en el offset ***0x3Ch*** hacia un valor incorrecto, lo que debemos hacer es reajustarlo para que aunte a la cadena PE, para esto miramos en que offset se encuentra, para esto seleccionamos como en la imagen la cabecera y con el botón derecho pinchamos en ***Dump in CPU***, luego allí en la ventana ***Dump*** la encontramos en el offset ***0x100h***:

```
Address   Hex dump                                          ASCII
00400100  50 45 00 00 4C 01 0D 00 19 5E 42 2A 00 00 00 00  PE..L♪..↓^B*....
00400110  00 00 00 00 E0 00 8E 81 0B 01 53 52 00 10 01 00  ....α.Äüø♂SR.►♫.
00400120  00 C0 7C 00 00 00 00 00 D0 50 A1 01 00 A0 A0 01  .└|.....$Pí♫.áá♫
00400130  00 B0 9D 01 00 00 40 00 00 10 00 00 00 10 00 00  .▒♥♪..@..►...►..
00400140  01 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00  ☺.......♦.......
```

Como hemos dicho antes es cuestión de cambiar este valor para que apunte a esta cadena:

```
Address   Hex dump                                          ASCII
00400030  00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00  .............♪♪..
00400040  BA 10 00 0E 1F B4 09 CD 21 B8 01 4C CD 21 90 90  ||►.♫▼┤.=↑☺♫L=↑ÉÉ
00400050  54 68 69 73 20 70 72 6F 67 72 61 6D 20 6D 75 73  This program mus
00400060  74 20 62 65 20 72 75 6E 20 75 6E 64 65 72 20 57  t be run under W
00400070  69 6E 33 32 0D 0A 24 37 00 00 00 00 00 00 00 00  in32..$7........
```

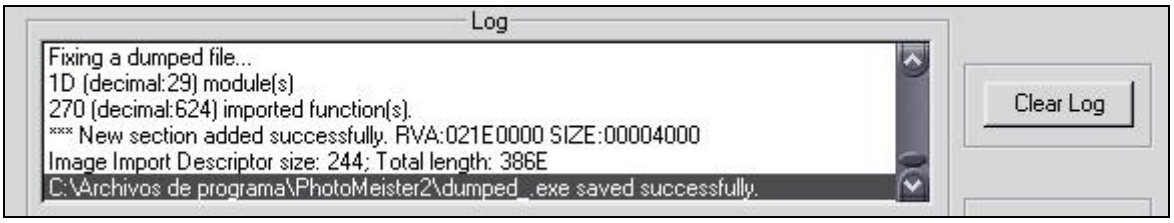
Escrito en rojo está el valor ***100h*** escrito al revés, es decir ***"00 01 00 00"*** y este tema está solventado, pero ahí no acaba todo, si seguimos mirando en la cabecera vemos que nos ha cambiado el número de secciones de ***12*** a ***13***:

```
00400100   50 45 00 00  ASCII "PE"        PE signature (PE)
00400104   4C01         DW 014C           Machine = IMAGE_FILE_MACHINE_I386
00400106   0D00         DW 000D            NumberOfSections = D (13.)
00400108   195E422A     DD 2A425E19        TimeDateStamp = 2A425E19
0040010C   00000000     DD 00000000        PointerToSymbolTable = 0
00400110   00000000     DD 00000000        NumberOfSymbols = 0
00400114   E000         DW 00E0            SizeOfOptionalHeader = E0 (224.)
```

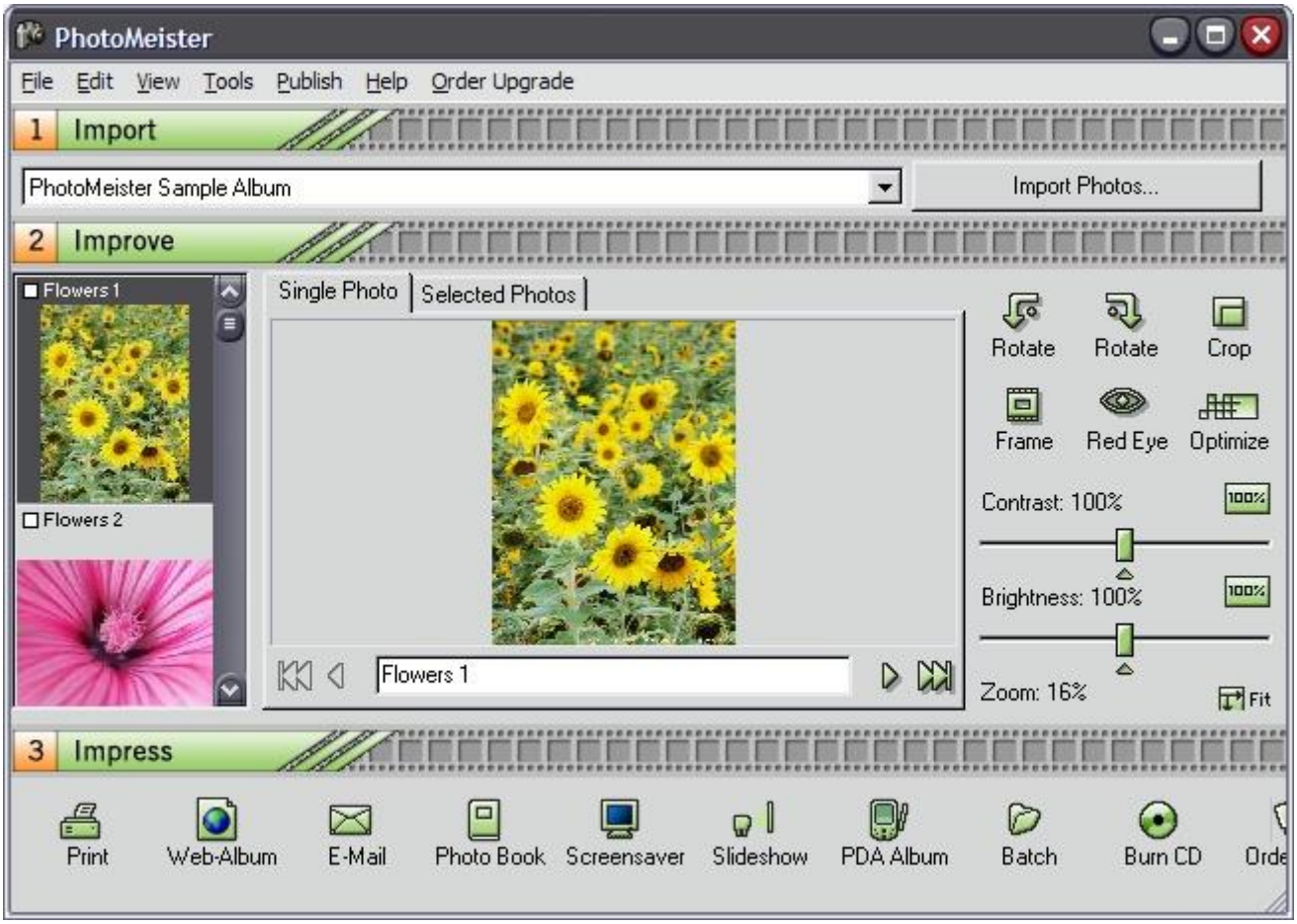
que se repara yendo al offset ***0x106h*** y cambiar el valor ***0Dh*** existente por un ***0Ch*** y ahora si como tenemos ***LordPE*** abierto volcamos el archivo con la opción ***Intellidump*** activada (importante esto), lo guardará por defecto como ***dumped.exe***, yo dejo este nombre.

Llegó el turno a ***Import Reconstructor*** (ImpRec), lo abrimos, vamos a ***Attach to an Active Process*** donde seleccionamos el proceso de ***photomeister2.exe***, le damos estos datos:

Pulsamos ***Get Imports*** y nos aparecerán todas las funciones importadas correctas, luego pinchamos ***Fix Dump*** y seleccionamos el archivo volcado ***dumped.exe***, nos dejará uno llamado ***dumped_.exe*** al lado que es el que debemos probar en cuanto termine veremos esto en la ventanita ***log***:

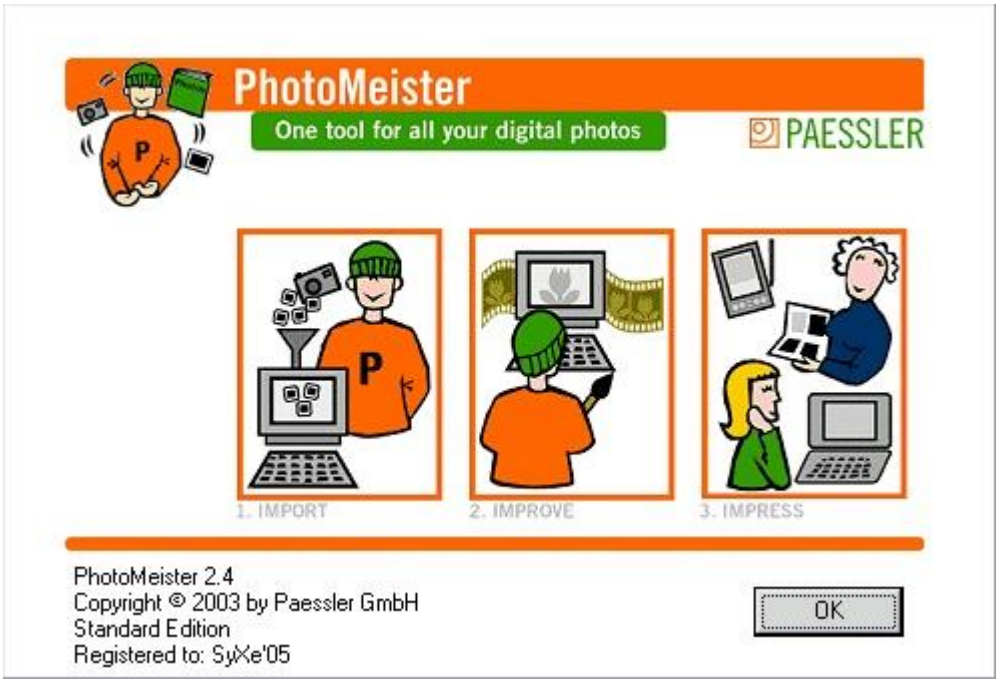


Nos está diciendo que el archivo ya se ha salvado, ahora lo probamos, doble clic sobre él y....



jeje.. allí está rulando, la verdad es que a la primera pocas veces va a suceder normalmente hay que arreglar algo como alguna llamada a ***GetEnvironmentVariableA***, o bien busca ***ArmAcces.dll*** y cositas de estas que en este caso nos ahorramos, lo que indica que hemos terminado con el desempaquetado.

Ahora que me doy cuenta hemos enrtado sin ver la nag ;DDD y mirad el About



Esta ventanita antes decía ***Trial Edition (30 Days Left)***, lo que me hace pensar que el ***Trial*** ha caído junto con ***Armadillo*** que es lo más normal ya que es él quien suele llevar la limitación temporal, yo almenos lo he visto así en otros programa, o sea que nos hemos cargado dos problemas de un solo plumazo, así da gusto eh!

Tenemos la opcion de registrarnos pero nos sacara el mensaje de error directamente debido a que no podemos validar el serial ya que ***ArmAcces.dll*** no la importa nuestro ***.exe*** y la necesita para esto. El que el About diga ***Registered To***: no debe confundirnos porque el programa sigue limitado, sino intentad quemar una foto en un ***CD*** vereis la enorme nag restrictora que no nos deja hacer la funcion, lo mismo ocurre con otras funciones.

Sin perder ni un segundo vamos al ***CommandBar*** y ponemos ***Bp ShowWindow***, picamos en ***Burn CD*** para que aparezca la nag en pantalla, antes claro saltara nuestra trampa, jeje.. deshabilitamos el ***BPX*** y vamos traceando hacia atras, llegamos a un loop en el que sale la nagscreen:

nag en bucle :456381 y :4563C1

ponemos un BPX al final de este y aceptamos, luego seguimos traceando hacia atras, pero esta vez no hay que esperar mucho para caer en esta interesantisima zona de codigo donde se decide todo:

```
:006EA9F8   E8 038FDBFF     CALL dumped_.004A3900        ;Tenemos permiso para realizar la funcion?
:006EA9FD   83F8 02         CMP EAX,2                    ;Si EAX=2 saltara..
:006EAA00   74 31           JE SHORT dumped_.006EAA33    ;Y si salta es que podemos hacer la funcion ;)
:006EAA02   33C9            XOR ECX,ECX
:006EAA04   B2 01           MOV DL,1
:006EAA06   A1 E0375B00     MOV EAX,DWORD PTR DS:[5B37E0]
:006EAA0B   E8 A074D6FF     CALL dumped_1.00451EB0
:006EAA10   8B15 48257100   MOV EDX,DWORD PTR DS:[712548]
:006EAA16   8902            MOV DWORD PTR DS:[EDX],EAX
:006EAA18   A1 48257100     MOV EAX,DWORD PTR DS:[712548]
:006EAA1D   8B00            MOV EAX,DWORD PTR DS:[EAX]
:006EAA1F   8B10            MOV EDX,DWORD PTR DS:[EAX]
:006EAA21   FF92 D8000000   CALL DWORD PTR DS:[EDX+D8]   ;Call_Restrictor que muestra la nag..!!
```

En este trozo de codigo se decide si mostrara la nag o bien realizara la funcion, todo depende del valor de retorno que de aqui se lleve, luego lo verificara y hara una cosa u otra en funcion del resultado obtenido, es por esto que de aqui se debe de llevar un ***EAX=2*** como una catedral GoTiKa ;XDD

Aqui me encontre una cosa muy curiosa: Si entraba en el ***call 4a3900*** y lo forzaba a retornar ***EAX=2*** ya que es el valor esperado resulta que no hacia la funcion y seguia sacando la nag, aun llevandose un trorno valido..!!! esto me confundio bastante al principio, pense que estaba haciendo algo mal o que quiza aplicaba el parche en un mal lugar, no se muchas cosas, pero no se porque lo hice pero parchee el call y guarde los cambios un.exe con otro nombre como siempre para diferenciarlo, desues ejecute este otro exe, le dije quemar un ***CD*** y...

Pues eso que la dejaba hacer...!! no se si es una trampa para despistar pero conmigo ciertamente casi lo consigue, pues no se porque pasa esto, quiza la rutina sea llamada desde un principio y el retorno lo guarde en un ***FLAG*** que despues verifique y compare con los sucesivos retornos, en caso de no coincidir sabe que alguien esta husmeando y no realiza la funcio, o bien es un chequeo de estos o no lo entiendo pero esto me mosqueo mucho y no fue facil de averiguar, pero bueno ya tenemos el parche ;DDD

Para que el call 4a3900 retorne EAX=2 cambiaremos (un parche sencillo): veamos el final del call:

```
:004A3B29   8D45 D4          LEA EAX,DWORD PTR SS:[EBP-2C]
:004A3B2C   BA 0A000000      MOV EDX,0A
:004A3B31   E8 2E05F6FF      CALL dumped_.00404064
:004A3B36   C3               RETN
:004A3B37 ^ E9 18FFF5FF      JMP dumped_.00403A54
:004A3B3C ^ EB EB            JMP SHORT dumped_.004A3B29
:004A3B3E   8B45 FC          MOV EAX,DWORD PTR SS:[EBP-4]
```

Debemos modificar el codigo para que quede EAX=2 ya que un simple AL=2 no nos vale, pero el problema es que nos faltan bytes porque la instrucción que podemos modificar está en :4a3b3e y son solo tres bytes, para poner EAX a 2 necesito almenos 4 (XOR EAX, EAX +2 x INC EAX) que supone 33C04040, esto es un problema porque a la salida del call se verifica el valor de EAX completo y no el de AL como suele ser lo habitual, pero no desesperemos, un poco más arriba tenemos un LEA EAX, DWORD PTR SS:[EBP-2C] que tiene todos los números para ser modificado, ya que lo puse a 0 como prueba y el call posterior funcionó bien por lo que deduje que no habría problema si modificaba la instrucción citada, entonces abreviando podríamos cambiar el código anterior por este otro y probarlo:

```
:004A3B29   33C0             XOR EAX,EAX          ;EAX=0
:004A3B2B   40               INC EAX              ;EAX:=EAX+1 => EAX=1
:004A3B2C   BA 0A000000      MOV EDX,0A
:004A3B31   E8 2E05F6FF      CALL dumped_.00404064
:004A3B36   C3               RETN
:004A3B37   E9 18FFF5FF      JMP dumped_.00403A54
:004A3B3C   EB EB            JMP SHORT dumped_.004A3B29
:004A3B3E   40               INC EAX              ;EAX:=EAX+1 => EAX=2...!!!
:004A3B3F   90               NOP                  ;Nopeamos los bytes sobrantes..!!
:004A3B40   90               NOP
```

Salvamos los cambios, probamos el ***.exe*** y... ***NO FUNCIONA!!***

Por lo visto acepta el cambio en memoria pero no permanente, se me ocurre lo de siempre: un ***injerto***! pensé entonces que podría dejar al call que trabajara bien pero despues del ret saltar a una zona libre y allí ensamblar un ***MOV EAX, 00000002*** para luego retornar al punto de origen, con este cambio debería además nopear la modificacion de EAX en ***:4a3b3f*** para que no nos chafe el valor. Dichjo asi puede parecer dificil pero en realidad es super sencillo, primero buscaremos un lugar libre y que este a nuestro alcance, el salto tampoco puede ir muy lejosa aunque sea largo asi que mejor una zona cercna, necesitamos 5 bytes del MOV + otros 5 del salto (si es largo) qu serian 10 bytes, si el salto fuera corto con 7 tendriamos bastante, el lugar para injertar sera este:

```
:004A3B3E   8B45 FC          MOV EAX,DWORD PTR SS:[EBP-4]
```

si necesitamos mas de los bytes porque el salto sea largo los cogemos y luego ejecutamos alli las instrucciones sobrescritas antes de retornar y ya esta, esto no es problema, busquemos un lugar con algunos bytes libres, con ceros por ejemplo: Vaya Regalo! tenemos un gran ristra de ceros entre :4a3770 y :4a386c

Necesitamos un salto largo porque uno corto no nos alcanza, entonces las instrucciones que necesitamos son:

```
:004A3B3E   8B45 FC          MOV EAX,DWORD PTR SS:[EBP-4]
:004A3B41   5F               POP EDI
:004A3B42   5E               POP ESI
```

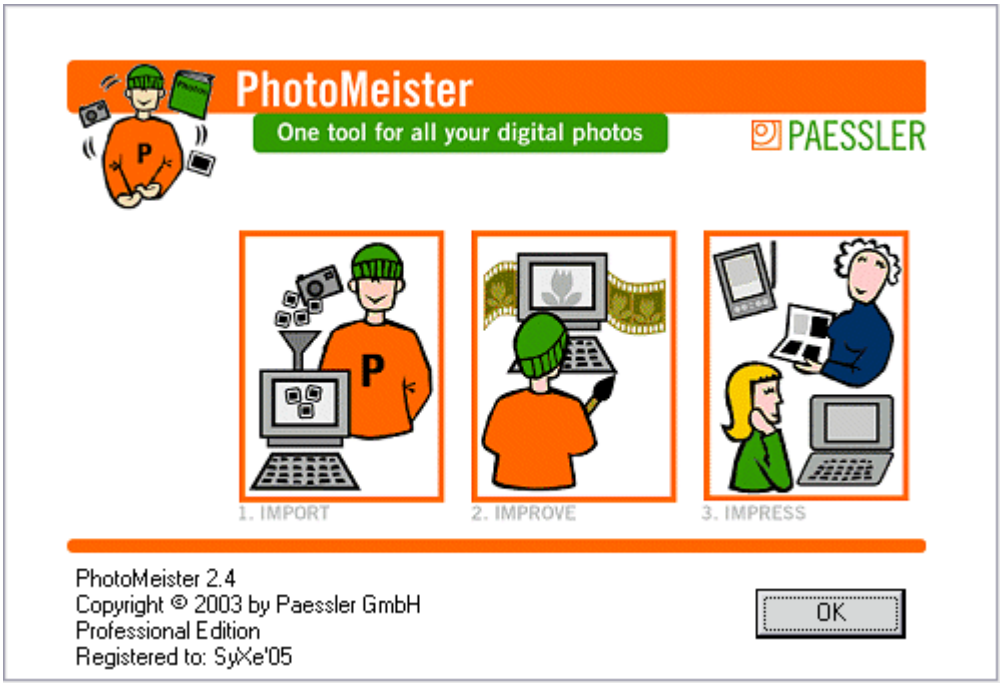
Las tres las vamos a sobrescribir, el MOV EAX.. no imoprta pero las otras dos hay que ejecutarlas en el propio injerto y luego saltaremos directamente a :4a3b43 pero ya con EAX=2, jeje.. veamos como quedara el salto:

```
:004A3B3E   E9 BDFCFFFF       JMP dumped_.004A3800
```

Ahora vamoa a :4a3800 y ensamblamos esto:

```
:004A3800   B8 02000000       MOV EAX,2
:004A3805   5F                POP EDI
:004A3806   5E                POP ESI
:004A3807   E9 37030000       JMP dumped_.004A3B43
```

Y ya tenemos el injerto listo, seleccionamos los bytes del propio injerto y hacemos un ***"Copy to Executable -> All modifications"*** luego elegimos en la nueva ventanita ***"Save File"*** le damos un nombre y a correr... si no nos deja salvar todo de golpe pues no pasa nada salvamos la mitad, abrimos el archivo medio modificado y lo terminamos, luego lo probamos y mirad...



tenemos la ***Versión Profesional*** ;DDD está claro que el injerto ha funcionado y bastante bien, ahora si podemos estar tranquilos porque todas las opciones y funciones del programa estarán operativas ***100%*** tambien veiamos antes algunos indicadores de la version ***Trial*** que han desaparecido como el boton del menu ***Order Upgrade*** y algunas cosillas mas.

Y con esto damos por finalizado el tuto, espero que os haya gustado, pronto volveremos con otros, si alguien quiere contactar conmigo por cualquier tema, ya sea una duda, una sugerencia o como no una crítica siéntase libre de escribirme a cualquiera de estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com